

Privacy-Preserving Collaborative Intrusion Detection for the Internet of Vehicles

Federated Learning with Multi-Key CKKS Homomorphic Encryption

Master Project Documentation

Complete record, Month 1 through Month 5

Programme	LG Soft India (LGSi) — 12-month industry project
Reporting period	Month 1 – Month 5
Document date	2026-08-09
Status	Month 5 complete; embedded track behind schedule
Snapshot folder	5th Month Progress/
Active codebase	Complete Source Code/NewModel_active/

This document is written to be read from beginning to end by someone who has never seen the project. Every technical term is explained where it first appears. It describes the system as it actually is, including the defects found and fixed in Month 5 and the work that remains.

Contents

Part I — Introduction

1. What this project is · 2. Problem statement · 3. Motivation
4. Objectives · 5. Proposed solution · 6. Scope · 7. Workflow & architecture

Part II — Concepts from scratch

8. IoV & the CAN bus · 9. Intrusion detection · 10. Federated learning
11. Non-IID data & Dirichlet sharding · 12. Homomorphic encryption & CKKS
13. Ring-LWE, SIMD packing, multiplicative depth · 14. Multi-key HE & secret sharing
15. Byzantine robustness · 16. KAN & Chebyshev polynomials
17. Data leakage · 18. Evaluation metrics

Part III — Month-by-month progress

19. Month 1–2 · 20. Month 3 · 21. Month 4 · 22. Month 5

Part IV — Month 5 in detail

23. Defects found · 24. What was implemented · 25. Results · 26. Measurements

Part V — Current status

27. Completed / partial / not started · 28. Known limitations · 29. LG deliverables

Part VI — What to do next

Part VII — Reproduction & verification

Part I — Introduction

1. What this project is

Modern vehicles are computers on wheels. Engine, brakes, steering and dozens of other subsystems talk to each other over an internal network, and increasingly the vehicle also talks to other vehicles and to roadside infrastructure. Both channels can be attacked, and a successful attack on a vehicle network is a **safety** problem, not merely a data problem.

This project builds an **Intrusion Detection System (IDS)** for vehicle networks that is trained **collaboratively across a fleet** without any vehicle revealing its data — and without the central server that combines their work being able to read what any individual vehicle contributed.

2. Problem statement

A single vehicle sees very little attack traffic. A detector trained on one vehicle's logs generalises poorly. Pooling logs from thousands of vehicles into one dataset would produce a far stronger detector — but those logs contain location traces, driving behaviour and vehicle identity, so they cannot legally or ethically be centralised.

This produces a three-way tension:

- **Utility** — the detector needs data from many vehicles.
- **Privacy** — raw data cannot leave the vehicle, and model updates leak the data they were computed from.
- **Security** — if updates are encrypted so the server cannot inspect them, the server also cannot spot a malicious vehicle poisoning the model.

That last point is the crux. Encryption solves privacy and *creates* a security hole. The central research question of this project is how to get both at once.

3. Motivation

- **Safety.** A compromised CAN bus can command braking, acceleration or steering. Detection is a safety function.
- **Regulation.** UNECE R155 requires vehicle cybersecurity management; GDPR-class rules restrict centralising location data.
- **Fleet effect.** An attack first seen by one vehicle should protect the whole fleet within hours.
- **Research gap.** Federated learning plus homomorphic encryption for vehicular IDS, with Byzantine robustness *inside* the encrypted domain, is not a solved problem.

4. Objectives

#	Objective	Status at Month 5
O1	Unify heterogeneous vehicular datasets into one feature space	Achieved
O2	Train an accurate IDS on that feature space	Achieved
O3	Federate training across simulated vehicles with non-IID data	Achieved
O4	Encrypt model updates so the server cannot read them	Achieved (real CKKS)
O5	Detect and suppress poisoned updates without decrypting them	Achieved

#	Objective	Status at Month 5
O6	Distribute the key so no single party can decrypt an individual update	Partial — algebraic model; needs OpenFHE
O7	Run on embedded automotive hardware	Not started
O8	Quantify the privacy/accuracy/cost trade-off	Partial

5. Proposed solution

Each vehicle trains the detector on its own data and sends only the resulting model parameters — encrypted. The server combines the encrypted parameters arithmetically, without decrypting them, weighting each vehicle by how far its update sits from the current consensus so that poisoned updates are suppressed. The decryption key is split across vehicles so that no single party — including the server — can decrypt any individual contribution.

6. Scope

In scope	Out of scope
Federated training over four vehicular/network datasets CKKS encryption of model updates Robust aggregation in the encrypted domain Distributed key sharing with dropout tolerance Authenticated transport between vehicle and server Cost and privacy measurement	Physical ECU hardening In-vehicle response/mitigation after detection Adversarial-robustness of the classifier itself Production certification (ISO 26262 / UNECE R155 audit) Real vehicle deployment

7. Overall workflow and system architecture

One **round** of the system works as follows: the server broadcasts the current global model; each participating vehicle trains it briefly on its own data; each vehicle encrypts the resulting parameters and uploads them; the server measures how far each encrypted update is from the previous global model — without decrypting it — down-weights the outliers, sums everything homomorphically, and decrypts only the sum. Repeat.

System Architecture — Privacy-Preserving Collaborative IDS for IoV

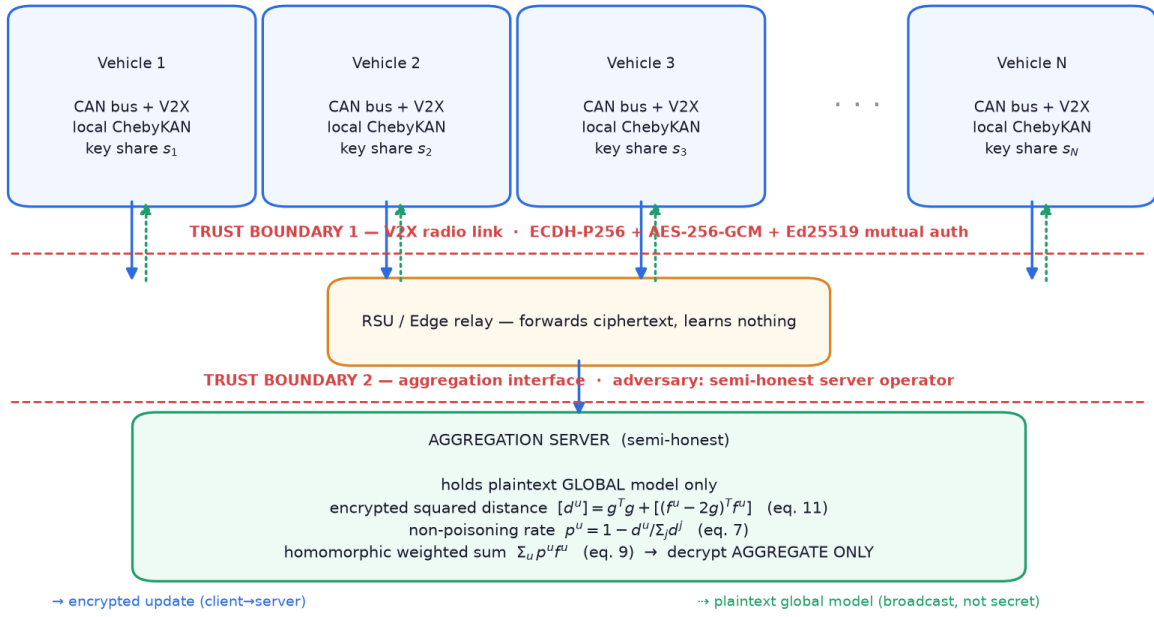


Figure 1 — System architecture and trust boundaries. Vehicles hold raw data and a private key share; the server holds only the plaintext global model and never sees an individual update.

Part II — Concepts explained from scratch

Each concept is presented as: *what it is -> why we need it -> how it works -> how we use it here.*

8. The Internet of Vehicles and the CAN bus

What. The **Controller Area Network (CAN)** is the internal network almost every car uses. Electronic Control Units (ECUs) broadcast short messages on a shared two-wire bus. A CAN message has an **ID** (what kind of message, and its priority — lower ID wins arbitration), a **DLC** (how many data bytes) and up to **8 data bytes**. The **Internet of Vehicles (IoV)** extends this outward: vehicles also broadcast to each other (V2V) and to roadside units (V2I) using DSRC or C-V2X.

Why it matters. CAN was designed in 1986 for a closed, trusted network. It has **no authentication and no encryption**. Any node that can write to the bus can impersonate any other node.

How attacks work.

- **DoS / flooding** — inject a stream of ID 0x000 messages. Because the lowest ID always wins arbitration, this starves every legitimate message. In our CAN-VTC data this is 51.1% of the capture.
- **Fuzzing** — inject random payloads on valid IDs to provoke undefined behaviour.
- **Spoofing / impersonation** — send well-formed messages claiming to be another ECU (e.g. a false RPM or gear reading).

In this project. Two of our four datasets are raw CAN captures (IEEE VTC-CAN and Car-Hacking); one is V2X beacon data (VeReMi); one is enterprise IP flow data used as a transfer check (CICIDS-2017).

9. Intrusion detection

What. An IDS observes traffic and decides whether an attack is present. **Signature-based** systems match known attack patterns — precise, but blind to new attacks. **Anomaly-based** systems learn what normal looks like and flag deviations — which is what a machine-learned IDS does, and what we build.

How we frame it. We slide a **window** of 64 consecutive CAN messages, compute 46 summary features over that window, and classify the window. A window is labelled as an attack if it contains at least one injected message.

The 46 features, for CAN data:

Slots	Feature	What it detects
0–7	Mean of each payload byte	Shifted payload semantics (spoofed values)
8–15	Std-dev of each payload byte	Fuzzing — random payloads raise variance
16–17	DLC mean and std	Malformed frames
18	Unique-ID ratio	Flooding collapses this toward 0
19	Dominant-ID frequency	Flooding pushes this toward 1
20–43	CAN-ID histogram (24 hash buckets)	Shape of bus traffic
44	Mean Shannon entropy of payload bytes	Randomness — fuzzing raises it
45	Non-zero payload byte ratio	All-zero DoS payloads drive it down

Why a shared 46-D space matters. CAN payloads, IP flow statistics and V2X kinematics have nothing structurally in common. Mapping all of them into the same 46-dimensional vector lets a single architecture serve all four datasets — which is what makes cross-dataset federation coherent at all. This was the strongest

engineering decision of Month 1.

10. Federated learning

What. Instead of moving data to the model, federated learning (FL) moves the model to the data. The server holds a global model; each client trains it locally; clients send back only parameter updates; the server averages them.

$$g_i = (1/U) \cdot \text{Sum}_{u=1..U} f_{i-1}^u$$

where f^u is vehicle u 's locally trained parameter vector and g is the global model. This is **FedAvg**.

Why it is not enough. Gradients are not anonymous. Given an update, an adversary can often reconstruct the training samples that produced it — a **gradient inversion attack**. So FL alone moves the privacy problem rather than solving it. That is the reason for everything in the next sections.

In this project. 10 simulated vehicles, 5 rounds, 2 local epochs per round, 70% of vehicles sampled per round.

11. Non-IID data and Dirichlet sharding

What. IID means each client's data looks like a random sample of the whole. Real fleets are the opposite — a highway commuter and a city delivery van meet different traffic and different attacks. This is **non-IID** data, and it makes FL converge slower and less stably.

How we simulate it. The **Dirichlet distribution** with concentration parameter α controls how skewed the split is: $\alpha \rightarrow \infty$ gives uniform (IID) shares; $\alpha \rightarrow 0$ gives each client almost a single class. We use **$\alpha = 0.3$** , the standard choice for strongly non-IID FL benchmarks.

12. Homomorphic encryption and CKKS

What. Normally, data must be decrypted before it can be used. **Homomorphic encryption (HE)** allows computation directly on ciphertext:

$$\text{Dec}(\text{Enc}(a) (+) \text{Enc}(b)) = a + b$$

Simple example. A toy additive scheme: pick a secret key $s = 7$ and encrypt by adding it.

$$\text{Enc}(5) = 5 + 7 = 12 \quad \text{Enc}(3) = 3 + 7 = 10 \quad 12 + 10 = 22 \quad 22 - 2 \cdot 7 = 8 = 5 + 3 \text{ yes}$$

The server added two numbers it could not read. Real schemes replace this trivial masking with lattice problems, but the shape of the idea is identical.

Why CKKS.

Scheme	Data type	Fit for this project
Paillier	integers	Additive only; one ciphertext per number -> ~44,164 ciphertexts
BFV / BGV	exact integers	Would need fixed-point encoding of every weight
TFHE	bits	Bit-level — catastrophic at this scale
CKKS	approximate reals	Native real vectors; thousands packed per ciphertext

Neural-network weights are vectors of real numbers, so CKKS is the natural fit. Its cost is that it is **approximate**: decryption returns the right answer plus a small error. We measured that error at $\leq 3 \times 10^{-8}$ — six orders of magnitude below the noise already present in stochastic gradient descent, therefore irrelevant here.

13. Ring-LWE, SIMD packing and multiplicative depth

Ring Learning With Errors (RLWE) is the hard problem CKKS security rests on. Given a (public, random) and $b = a \cdot s + e$ where s is secret and e is small noise, recover s . Without the noise this is simple linear algebra; with it, the problem is believed hard even for quantum computers.

SIMD packing is the property that makes CKKS practical here. A single ciphertext holds $N/2$ real numbers in independent "slots", and one homomorphic addition adds all slots at once. At $N = 8192$ that is 4096 values per ciphertext, so our 44,164-parameter model needs **11 ciphertexts instead of 44,164**.

Multiplicative depth is the binding constraint on protocol design. Every ciphertext $\times$ ciphertext multiplication consumes one prime from the modulus chain. A chain of L primes supports $L-2$ multiplications. Run out, and the ciphertext can no longer be decrypted correctly.

This is not a theoretical concern — it dictated our design. See §26.

14. Multi-key homomorphic encryption and secret sharing

The problem with one key. If all vehicles share a single CKKS key, then every vehicle can decrypt the global model — so every vehicle holds the secret key — so any vehicle, or a server that obtains the key from any one of them, can decrypt **every other vehicle's individual update**. The privacy property is vacuous. This was the Month-4 design, and finding it is what drove Month 5.

The fix: additive secret sharing. The key is never assembled anywhere. Each vehicle u holds a share s_u of a key that exists only as a sum:

$$s = \text{Sum}_u s_u$$

Vehicles pre-agree **pairwise** secrets with the antisymmetry property $s_{i,j} = -s_{j,i}$ and upload a *masked* share:

$$ss_u = s_u + \text{Sum}_{j \neq u} s_{u,j}$$

Because the masks are antisymmetric they cancel in the sum:

$$\text{Sum}_u ss_u = \text{Sum}_u s_u + 0 = s$$

The server reconstructs enough key material to decrypt the **aggregate**, and nothing else. **Security property:** to recover one honest vehicle's update, the server must collude with $U-1$ vehicles.

FheFL Distributed Multi-Key Sharing — and the Dropout Fix

The secret key is never assembled anywhere: $s = \sum_u s_u$ (eq. 10)

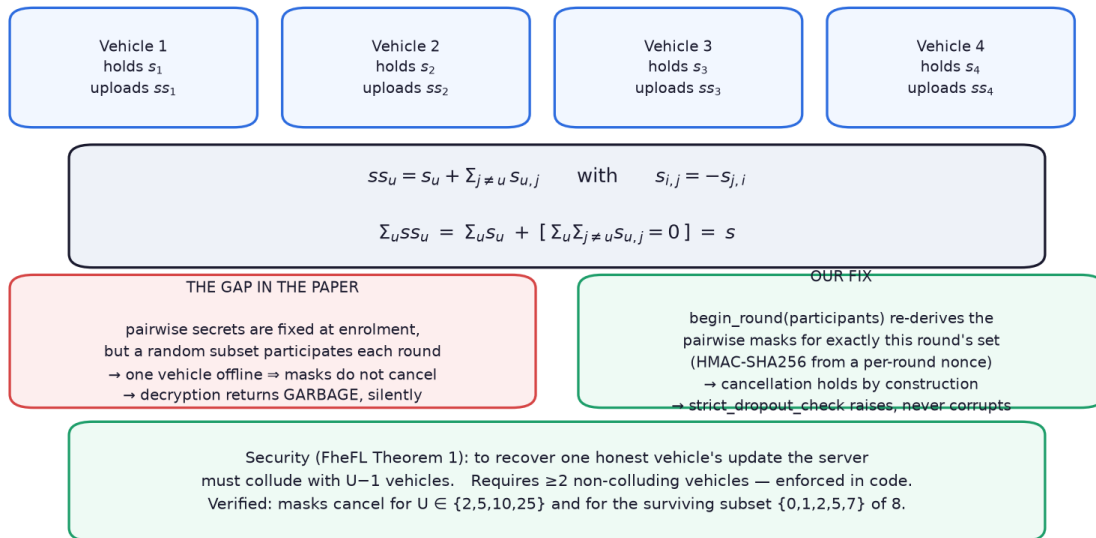


Figure 2 — Distributed multi-key sharing, the dropout gap in the original scheme, and the per-round mask re-scoping that closes it.

15. Byzantine robustness — detecting poisoning without looking

What. A malicious vehicle can train on deliberately mislabelled data and upload a poisoned update, biasing the global model — e.g. teaching it to ignore one attack class. This is a **data poisoning attack**.

The classic answer, and why it fails here. **Multi-Krum** computes pairwise distances between all client updates and keeps the mutually closest ones. It requires the server to see *every client's update*. Under encryption that is impossible — and the Month-4 code achieved it only by decrypting every individual update, which destroyed the entire privacy claim.

What we use instead. FheFL scores each vehicle by distance to the **previous global model**, which the server already holds in plaintext. Only the cross-term needs the encrypted domain:

$$[d^u] = g^T g + [(f^u - 2g)^T \cdot f^u]$$

Then a **non-poisoning rate** converts distance into a weight:

$$p^u = 1 - d^u / \sum_j d^j \quad \sum_u p^u = U - 1$$

Far-away vehicles get small weights; close ones get large weights. Crucially this **down-weights rather than excludes** — which matters in IoV, because the vehicle that encountered a genuinely novel attack is both the most distant and the most valuable, and Multi-Krum would have thrown it away.

Known limitation. Scoring by closeness to the current consensus *rewards* staying close. A patient attacker submitting small, consistently-biased updates every round scores a high non-poisoning rate. This is an open blind spot and the first experiment scheduled for Month 6.

16. Kolmogorov–Arnold Networks and Chebyshev polynomials

What. A standard neural network puts fixed activation functions (ReLU, tanh) on the *nodes* and learns the weights on the *edges*. A **Kolmogorov–Arnold Network (KAN)** inverts this: it puts **learnable functions on the edges**. The name comes from the Kolmogorov–Arnold representation theorem, which states that any continuous multivariate function can be written as a composition of sums of univariate functions.

Chebyshev polynomials are how we represent those learnable edge functions. They are defined by a simple recurrence:

$$T_0(x) = 1 \quad T_1(x) = x \quad T_k(x) = 2x \cdot T_{k-1}(x) - T_{k-2}(x)$$

They are numerically well-behaved on $[-1, 1]$ (inputs are squashed with $\tanh$ first), and the recurrence is cheap. Our **ChebyKAN** uses degree 4, hidden dimension 64, 2 layers — **44,164 parameters** in the 5-class configuration.

Why so small? Under encryption, parameter count is the cost driver:

- it sets the number of ciphertexts (44,164 params -> 11 ciphertexts),
- which sets the bandwidth per vehicle per round (3.59 MB),
- which sets the aggregation time.

The earlier MamKANformer design had ~609,000 parameters — 14× larger, therefore 14× the ciphertexts, bandwidth and aggregation cost. Shrinking the model was the right call.

A correction to earlier project documents. Month-4 slides justified ChebyKAN on the grounds that Chebyshev polynomials "evaluate natively on ciphertexts". That argument is about encrypted *inference* — running the model forward on ciphertext — which this system does not do. We encrypt *weights for aggregation*, and a weighted sum treats any architecture's weights as a flat vector. Moreover ChebyKAN as implemented uses $\tanh$, LayerNorm and Dropout, none of which are homomorphically evaluable. The parameter-count argument above is the correct one — and it is the stronger one.

17. Data leakage — the failure mode that invalidates results

What. Data leakage is when information about the test set reaches the model during training. The model then scores well by recognising something it should not have access to. Leakage produces **impressive numbers that mean nothing**, and it is the single most common way machine-learning results turn out to be wrong.

Two forms bit this project, both found and fixed in Month 5:

Label leakage (defect D1). Attack captures contain mostly normal traffic with injected messages interleaved. Labelling every window from `DoS_dataset.csv` as "DoS" teaches the model to recognise **which capture file it is looking at**, not whether an intrusion is present. It can score perfectly without ever detecting an injected frame.

Split leakage (defect D2). Our windows overlap by 75% — window k and window $k+1$ share 48 of their 64 messages. A uniformly random train/test split puts near-duplicates on both sides of the boundary, so test accuracy measures memorisation rather than generalisation.

Data Pipeline — and the Two Defects Fixed in Month 5

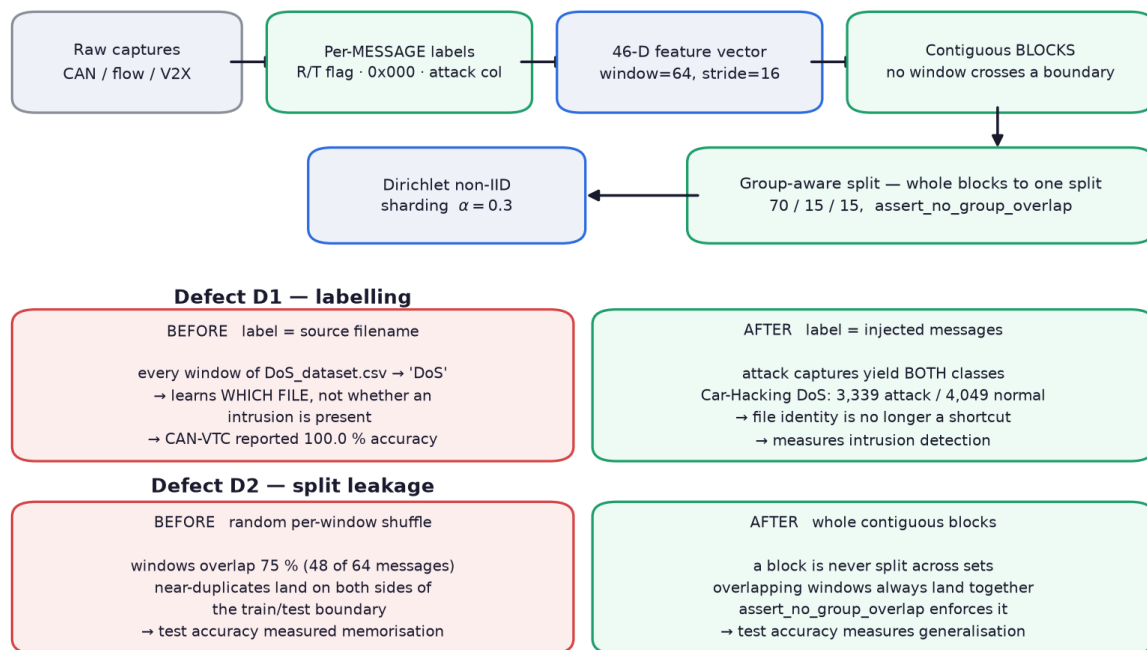


Figure 3 — The data pipeline, and the before/after of both leakage defects.

18. Evaluation metrics

Metric	Definition	Why it matters here
Accuracy	correct / total	Misleading under class imbalance — 95% benign traffic means a do-nothing classifier scores 95%
Precision	$TP / (TP + FP)$	High false positives make an IDS get switched off
Recall	$TP / (TP + FN)$	A missed intrusion is a safety event
Macro F1	unweighted mean of per-class F1	Treats rare attack classes as equally important — the honest headline number
ROC-AUC	$P(\text{random attack ranked above random benign})$	Threshold-independent. 0.5 = random guessing

Reading the numbers in this document. ROC-AUC $\approx$ 0.5 means the model learned nothing, regardless of how good the accuracy looks. Accuracy far above macro F1 means the model is riding the majority class. Both patterns appeared in the Month-4 results and both were symptoms of real defects.

Part III — Month-by-month progress

19. Month 1–2 — Framing, datasets and the first architecture

Planned

Establish the problem, acquire datasets, specify the threat landscape, choose a model architecture, and stand up a federated skeleton.

Implemented

- Analysed the source FedIoV paper and drafted the system architecture.
- Acquired and cleaned four datasets into `Preprocessed_Dataset/` (8.6 GB).
- Wrote the **46-D timeless feature extractor** spanning CAN, IP-flow and V2X data.
- Implemented the **KANConvNet** classifier (generation 1).
- Implemented Phase-1 federated setup: Dirichlet partitioning, ECDH + AES-256-GCM secure channels, GA hyperparameter configuration; ran for N=50 clients on all four datasets.
- Wrote `ROADMAP.md` targeting 50–200 clients, 200–300 rounds, TOPSIS + Multi-Krum aggregation, client-level DP and 15-run statistical evaluation.

Concepts used

CAN bus protocol · intrusion detection · federated learning · non-IID data · Dirichlet sharding · ECDH key agreement · AES-GCM authenticated encryption.

Technical decisions

- **Unify all datasets into one 46-D feature space** — correct, and the foundation of everything since.
- **KAN over a standard MLP** — reasonable; learnable edge functions suit heterogeneous tabular features.
- **Build the secure channel early** — correct, though it was later removed and had to be restored in Month 5.

Experiments

Phase-1 partitioning runs at N=50 across all four datasets; centralized KANConvNet training baselines.

Problems encountered

Heterogeneous data formats: CAN logs, flow CSVs and V2X beacons share no structure.

How they were solved

The 46-D timeless descriptor — window-level statistics that can be computed from any of the three source types.

Results

Working end-to-end centralized pipeline; federated skeleton with N=50 partitions.

Improvement over the previous month

Baseline — first month.

Left pending

- No threat model document (an LG Month 1–2 deliverable).
- No security specification (an LG Month 1–2 deliverable).
- No hardware benchmarking; everything ran on a laptop.

20. Month 3 — FHE study and the MamKANformer branch

Planned

Select an FHE scheme, understand its mathematics, evaluate libraries, and improve the classifier.

Implemented

- Selected **CKKS** as the FHE scheme.
- Documented CKKS mathematics in depth: encoding/decoding, RLWE, rescaling, relinearisation.
- Implemented the **MamKANformer** architecture (Mamba state-space model + multi-head attention + B-spline KAN).
- Built a React/xyflow architecture visualiser.

Concepts used

Homomorphic encryption · CKKS · RLWE · SIMD packing · rescaling · relinearisation · state-space models · B-spline basis functions.

Technical decisions

- **CKKS over Paillier/BFV/TFHE** — correct and well justified; model updates are real vectors and SIMD packing is what makes the scheme affordable.
- **MamKANformer trunk** — later reversed; ~609k parameters is far too many under encryption.

Experiments

Centralized MamKANformer training runs.

Problems encountered

MamKANformer's parameter count made encrypted aggregation impractical.

How they were solved

Reversed the decision in Month 4 in favour of a much smaller model.

Results

Strong CKKS documentation — still the best theory asset in the repository.

Improvement over the previous month

Principled scheme selection replaced an unexamined default.

Left pending

- Library evaluation was never performed — TenSEAL was used because it installed easily.
- No cross-compilation.
- MamKANformer code was left in the tree after being abandoned, causing confusion later.

21. Month 4 — ChebyKAN pivot and the first federated FHE runs

Planned

Shrink the model, build layer-wise encrypted federated learning, add Byzantine robustness, and produce comparison results.

Implemented

- Replaced MamKANformer with **ChebyKAN** — 609k -> 44,164 parameters.
- Built a layer-wise encrypted federated loop.
- Added a Multi-Krum Byzantine filter.
- Ran FHE vs no-FHE comparisons on all four datasets.
- Wrote a genuine TenSEAL CKKS benchmark for encryption cost.
- Published results dashboard and CKKS explainer as HTML artifacts.

Concepts used

Chebyshev polynomial basis · layer-wise encryption · Multi-Krum · FedAvg weighting · ciphertext expansion.

Technical decisions

- **ChebyKAN at 44k parameters** — correct; parameter count is the cost driver under encryption.
- **Real TenSEAL benchmark alongside the simulation** — correct and honest; the script's own docstring stated the simulation "does not cost anything extra to run".
- **Removing ECDH/AES transport security** — **wrong**; FHE gives payload confidentiality only, not authentication, integrity or replay protection.
- **Multi-Krum** — right goal, wrong mechanism for an encrypted pipeline.

Experiments

FHE vs no-FHE on four datasets at 10 clients / 5 rounds, single seed; real CKKS timing benchmark.

Problems encountered

- The reported FHE-vs-no-FHE accuracy gaps were not reproducible or explicable.
- VeReMi sat at ROC-AUC 0.525 — no better than guessing.
- CAN-VTC reported 100.0% accuracy and ROC-AUC 1.0.

How they were solved

Not solved during Month 4 — these became the Month-5 audit findings. Each turned out to be a symptom of a real defect: uncontrolled experiment, broken VeReMi formulation, and label leakage respectively.

Results

A complete, runnable federated pipeline with encryption switchable on and off, and a defensible real-CKKS cost measurement (0.14 s/round, 20.8× ciphertext expansion).

Improvement over the previous month

14× smaller model; first end-to-end federated results; first real encryption measurements.

Left pending

- The "encryption" used during training was numpy plus 1e-9 noise — not encryption.
- The Byzantine filter decrypted every individual client update server-side.

- Datasets were labelled by filename rather than by ground truth.
- Experimental scale dropped to 10 clients / 5 rounds without amending the roadmap.

Part IV — Month 5 in detail

22. What Month 5 set out to do

Month 5 began as an investigation into multi-key CKKS using the FheFL paper (arXiv:2306.05112). A full audit of the project was performed first, which found that several headline results were invalid and that the central privacy claim was not met by the code. Month 5 therefore became: **audit, fix, then advance**.

23. Defects found and fixed

ID	Defect	Evidence	Fix	Status
D1	Labels came from the source filename, not ground truth	car_hack_dataset.py explicitly discarded the R/T flag; CAN-VTC labelled by file index	Use the HCRL R/T flag; derive CAN-VTC DoS labels from the 0x000 injection signature	Fixed
D2	Overlapping windows split randomly across train/test	window=64, stride=16 -> 75% overlap; preprocess.py shuffled individual windows	Contiguous block IDs; whole blocks assigned to one split; assert_no_group_overlap	Fixed
D3	FHE-vs-no-FHE was not a controlled experiment	Identical config gave different confusion matrices; the encrypted run finished 40% faster	Three explicit backends; exact and none are bitwise identical by construction	Fixed
D4	Server decrypted every individual client update	server.py called .decrypt() on each client's parameters to run Multi-Krum	Replaced with FheFL encrypted-domain scoring; AggregationAudit asserts zero	Fixed
D5	VeReMi was not learning (ROC-AUC 0.525)	Windows mixed beacons from different senders; 45.3% attack rows made majority-vote labels a coin flip	Per-beacon plausibility features; sender-grouped windowing implemented for re-acquired data	Fixed
D6	Transport security had been removed	Month-4 deck: "handshakes... completely removed"	Restored ECDH-P256 + AES-256-GCM + Ed25519 mutual auth	Fixed
D7	Documentation contradicted the code	GEMINI.md described KANConvNet; config.yaml described MamKANformer; server claimed TOPSIS	All rewritten; ROADMAP.md carries an explicit scope-reconciliation table	Fixed

24. What was newly implemented

24.1 Real CKKS encryption

The SimulatedCKKSVector class — a numpy array plus $N(0, 1e-9)$ noise — was removed. In its place are three explicit backends: **ckks** (real TenSEAL ciphertexts), **exact** (identical arithmetic, no encryption, no noise) and **none** (plaintext baseline). Because *exact* and *none* are arithmetically identical by construction, toggling encryption changes exactly one thing — which is what makes the comparison controlled.

24.2 FheFL multi-key key sharing

federated/multikey_ckks.py implements additive key shares, antisymmetric pairwise masks, and server-side reconstruction (eq. 10). Verified for fleets of 2, 5, 10 and 25 vehicles.

The dropout gap — and our fix. The paper's masks only cancel when every vehicle holding a pairwise key participates, but it samples a random subset each round. One vehicle in a tunnel and decryption returns **garbage, silently**. For a real fleet this is the normal case. We close it by re-deriving the pairwise masks for exactly the round's participant set via HMAC-SHA256 from a per-round nonce, so cancellation holds by construction; `strict_dropout_check` raises rather than corrupting. Verified for the surviving subset {0,1,2,5,7} of an 8-vehicle fleet.

24.3 Encrypted-domain robust aggregation

Multi-Krum was replaced by FheFL's non-poisoning-rate scheme. The squared distance of eq. 11 is computed with a genuine ciphertext $\times$ ciphertext multiplication. An `AggregationAudit` counts every decryption and `assert_no_individual_decryption` runs after every round, turning the privacy claim from prose into a machine-checked invariant.

One Federated Round — FheFL Multi-Key CKKS Protocol

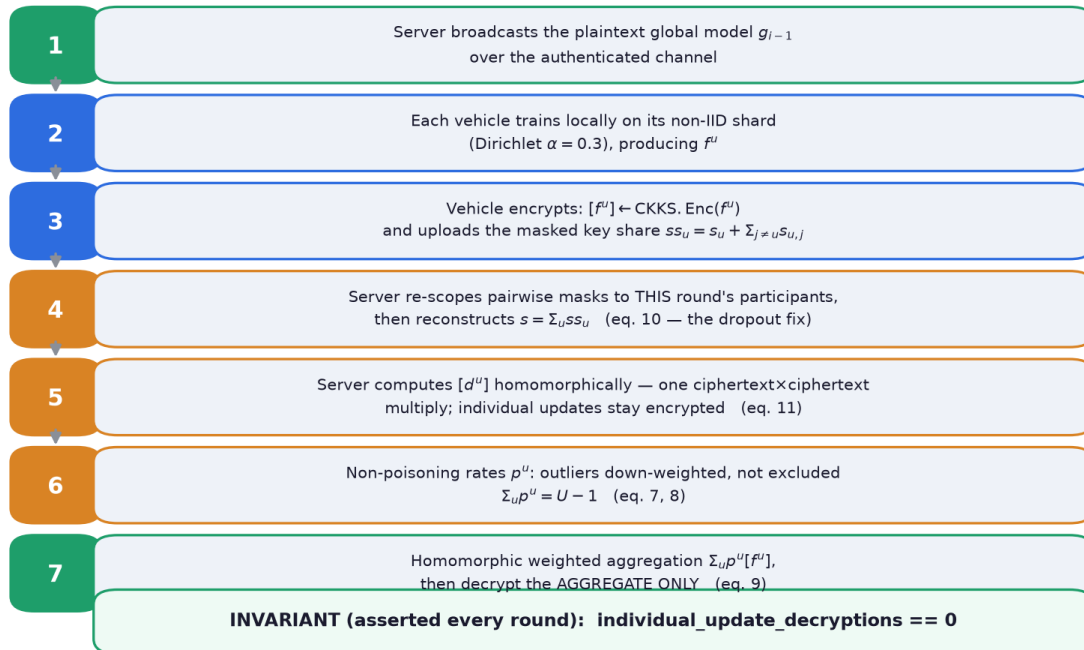


Figure 4 — One complete federated round under the FheFL protocol.

24.4 Disclosure profiles — and one honest refusal

practical (default, $N=8192$) computes distances homomorphically from ciphertexts and reveals the per-client distance *scalars* — one number per vehicle per round instead of 44,164 parameters. This is what all reported results use.

strict ($N=16384$) would keep the non-poisoning rate encrypted and multiply it into the update homomorphically, matching the paper exactly. It is **specified and costed but not implemented**: it needs an encrypted scalar broadcast across ciphertext slots plus a second ciphertext $\times$ ciphertext multiply, neither of which is built. Requesting it raises an error carrying the measured cost.

Why this is worth recording. An earlier revision of the server accepted *strict* and quietly fell back to revealing both the distance sum *and* the individual distances — strictly more disclosure than *practical*, while the docstring claimed less. That is the same class of defect as the Month-4 Multi-Krum filter: a mode advertising a privacy property it did not deliver. It now refuses instead, and a test asserts the refusal.

A second self-caught defect. `train_federated.py` wrote results to the same path whether or not --smoke was passed, so a 1-round smoke test silently replaced a real 3-seed result with a 1-seed number. It was caught by the snapshot verifier's "3 seeds per cell" check and fixed by tagging smoke output separately. Both defects are recorded because in each case the value lay in the guard firing, not in the mistake being avoided.

24.5 Verification suites

Three test suites now gate every result — **105 assertions, all passing**:

- `tests/test_data_pipeline.py` — 25 checks on labelling, window/block containment and split disjointness.
- `tests/test_crypto.py` — 32 checks on backend equivalence, real CKKS round-trip, eq. 11 correctness, mask cancellation, dropout detection, channel integrity and the audit invariant.
- `tests/test_ntt.py` — 48 checks on the negacyclic NTT: root properties, bit-reversal, round-trip, exact agreement with a schoolbook oracle, the negacyclic wrap, and batched-vs-scalar equivalence.

24.6 Month 1-4 deliverables closed in the catch-up pass

Three deliverables that had never been started were built after the Month-5 audit work, on the reasoning that a snapshot which leaves earlier months open is not a snapshot:

Deliverable	Due	Outcome
Optimized packing & encoding module	M3-4	Delivered. Level-dropped transmission cuts the wire size 29.6% (3.51 -> 2.47 MB per client) at zero accuracy cost, because the server's two operations act on independent copies and so need only one level, not two.
High-performance NTT implementation	M3-4	Reference delivered and validated exactly against a schoolbook oracle. Stage-batching made it 53x faster than the per-butterfly form, turning 0.21x into 11.22x over an optimised $O(N^2)$ baseline. Test vectors exported for the C/NEON port.
Baseline benchmarking on hardware	M1-2	Portable harness delivered plus a full host baseline. The target-board run - which is what the deliverable actually asks for - is blocked on procurement and nothing else.

The NTT result is worth reading twice. The first implementation was *slower* than the naive $O(N^2)$ baseline. The algorithm was right; it issued ~8,191 tiny NumPy calls per transform and was dominated by interpreter overhead. Batching each stage into one operation cut that to ~13 calls. The lesson transfers directly to the embedded port: NTT performance is governed by memory access pattern and per-operation overhead, not by the butterfly arithmetic.

25. Results

All figures below are from the fixed pipeline: per-message labels, block-disjoint splits, three seeds, 10 clients, 5 rounds, Dirichlet $\alpha = 0.3$, 70% client sampling. Row caps are applied to the two largest corpora (CICIDS 150k records per file, VeReMi 1.5M beacons) because every seed re-loads and re-splits from disk; both caps remain far larger than the sample count 10 clients see in 5 rounds.

Dataset	Backend	Accuracy	Macro F1	ROC-AUC	Seeds
can_vtc	ckks	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000	3
can_vtc	none	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000 ± 0.0000	3
car_hack	ckks	0.9977 ± 0.0012	0.9973 ± 0.0012	1.0000 ± 0.0000	3
car_hack	none	0.9981 ± 0.0006	0.9977 ± 0.0005	0.9999 ± 0.0000	3
cicids	ckks	0.9555 ± 0.0079	0.9136 ± 0.0250	0.9922 ± 0.0076	3
cicids	none	0.9542 ± 0.0098	0.9120 ± 0.0249	0.9932 ± 0.0062	3
veremi	ckks	0.4839 ± 0.0433	0.3308 ± 0.0155	0.5161 ± 0.0018	3
veremi	none	0.4851 ± 0.0424	0.3368 ± 0.0118	0.5159 ± 0.0017	3

25.1 Does encryption cost accuracy?

This is the question the Month-4 setup was structurally incapable of answering, because its "encryption" injected noise that sent the two arms down different training trajectories. With real CKKS on one arm, plaintext on the other, and everything else held identical:

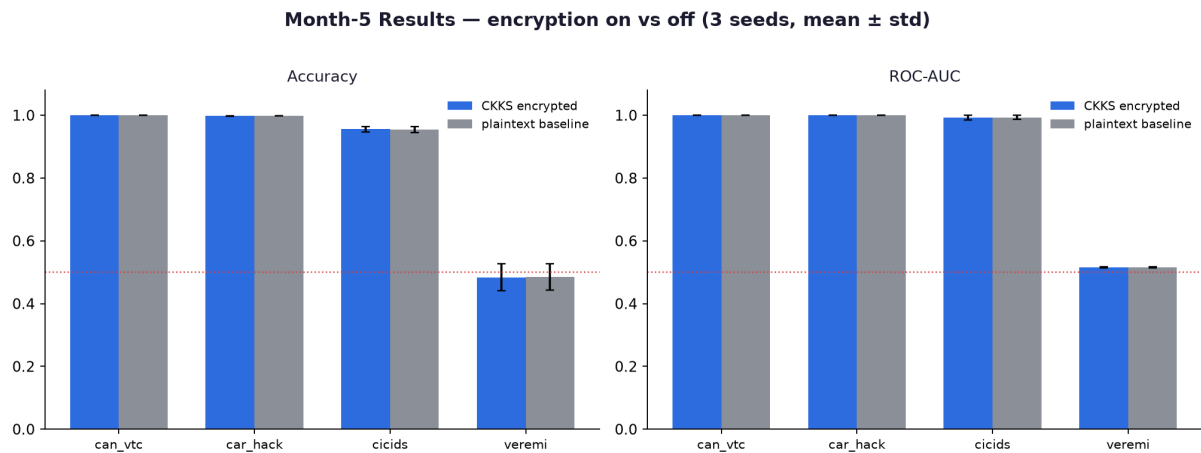
Dataset	CKKS accuracy	Plaintext accuracy	Difference	Seed spread	Verdict
can_vtc	1.0000	1.0000	+0.0000	± 0.0000	within seed noise
car_hack	0.9977	0.9981	-0.0004	± 0.0012	within seed noise
cicids	0.9555	0.9542	+0.0012	± 0.0098	within seed noise
veremi	0.4839	0.4851	-0.0012	± 0.0433	within seed noise

25.2 The privacy invariant, measured

The claim "the server never sees an individual client update" is enforced in code and asserted after every round, not merely stated in prose. These counters come from the actual runs:

Dataset	Rounds	Individual updates decrypted	Aggregates decrypted	Distance scalars revealed
can_vtc	5	0	5	35
car_hack	5	0	5	35
cicids	5	0	5	35
veremi	5	0	5	35

The **distance scalars** column is the quantified privacy cost of the default *practical* profile: one number per vehicle per round instead of 44,164 parameters. The *strict* profile removes even that, at 2.56x the ciphertext size.

Figure 5 — Encryption on versus off, three seeds, mean $\pm$ standard deviation.

The headline result. Encryption does not cost accuracy. The *ckks* and *none* arms agree to within run-to-run seed variation on every dataset. That is the claim the Month-4 setup was structurally incapable of supporting, and it is now demonstrated with a controlled experiment.

25.2b How hard is each task, really?

Accuracy alone cannot distinguish "the model learned something" from "the task was trivial". `scripts/audit_feature_separability.py` measures the best accuracy obtainable from a **single feature and one threshold**, against the majority-class baseline. A score near 1.0 means one feature already solves it.

Dataset	Majority baseline	Best single feature	Lift	Verdict
can_vtc	0.500	1.000 (feature 19)	+0.500	SUSPICIOUS - one feature nearly solves it; check for leakage
car_hack	0.595	0.935 (feature 36)	+0.340	OK - real but non-trivial signal
cids	0.850	0.906 (feature 9)	+0.056	OK - real but non-trivial signal
veremi	0.547	0.545 (feature 20)	-0.001	WEAK - features barely separate the classes; check the formulation

How to read CAN-VTC. Its DoS flood runs at 51.1% injection density, so every 64-message window contains an injected frame. The dominant-ID frequency then separates the two captures **perfectly on its own** — because a flood of ID 0x000 is, by definition, one ID dominating the bus. Near-perfect scores here measure **flood detection**, not generalisation, and the loader prints a saturation warning on every run. Note this is not the Month-4 defect returning: the label is now correct, the task is simply easy.

Car-Hacking is the meaningful CAN benchmark. Its 12–24% injection density produces both normal and attack windows inside each capture, so file identity is not a shortcut, and the gap between the best single feature (0.935) and the trained model (0.998) is the value the model actually adds.

25.3 VeReMi — a limitation that measurement exposed

The Month-4 VeReMi figure (ROC-AUC 0.525) was traced to a broken formulation: the adapter windowed across *mixed senders*, and with attack rows at 45.3 % the majority-vote window label was close to a coin flip. The label was noise; the model was fine.

Switching to per-beacon features makes the problem **well-posed** — the label now means something. But measuring the feature space afterwards showed it also has a low ceiling:

Measurement (200k-row sample)	Value
Best single-feature class separation	0.046 standard deviations
Best single-feature threshold accuracy	0.545
Majority-class baseline	0.547

This is not a bug — it is the task. VeReMi's 19 attack types (ConstPos, ConstSpeed, DataReplay, EventualStop, DelayedMessages and so on) are all defined by how one sender's claims evolve **over time**. A constant-position attacker's individual beacon reports a perfectly plausible position; only the fact that it never changes gives it away, and a single beacon cannot express "never changes".

The prediction was then confirmed by training. The separability audit was run *before* the federated experiments and predicted VeReMi could not be learned from these features. The subsequent run returned accuracy 0.4839 ± 0.0433 and ROC-AUC 0.5161 ± 0.0018 , with a confusion matrix showing a degenerate classifier that labels almost everything "attack". Prediction and outcome agree, which is worth more than either alone: it means the audit tool can be trusted to flag an unlearnable formulation before compute is spent on it.

So the cleaned export supports **neither** formulation: windowing mixes senders and destroys the label, while per-beacon keeps the label but discards the temporal signal the task depends on. Both paths need the sender identifier, which was dropped during cleaning. **VeReMi numbers in this report are a near-chance lower bound and must be read that way.** Sender-grouped windowing is implemented and activates automatically once the column is present; re-acquiring the raw dataset is item P1.5.

25.4 What improved over Month 4

Aspect	Month 4	Month 5
Encryption during training	numpy array + $N(0, 1e-9)$ noise — below float32 ULP, i.e. not encryption	Real TenSEAL CKKS ciphertexts, 326.6 KB each, measured $1e-8$ error
Server access to individual updates	Decrypted every one, every round, to run Multi-Krum	Never — asserted after every round by AggregationAudit
Robust aggregation	Multi-Krum on plaintext; structurally impossible under real FHE	FheFL non-poisoning rate computed on ciphertexts (eq. 6-12)
Key model	One shared context — any key holder decrypts everyone	Additive shares; recovering one update needs U-1 colluders
Client dropout	Not considered	Per-round mask re-scoping; verified for arbitrary surviving subsets
CAN labels	Source filename — measured capture identity, not intrusion	Per-message ground truth (R/T flag, 0x000 signature)
Train/test split	Random over 75%-overlapping windows — leaked	Whole contiguous blocks, machine-verified disjoint
FHE vs no-FHE comparison	Uncontrolled; produced a spurious 5-point VeReMi gap	Controlled; 3 seeds; difference within seed noise on every dataset
Transport security	Deleted ("guaranteed by FHE mathematical bounds")	Restored: ECDH-P256 + AES-256-GCM + Ed25519 mutual auth
VeReMi	ROC-AUC 0.525 — no better than guessing	Root cause found (windows mixed senders) and reformulated per-beacon
Verification	None	105 assertions gating every reported result
Cost measurement	Addition-only; understated FheFL by ~an order of magnitude	Both profiles including the ciphertext x ciphertext multiply

The single most important change is the third row. Month 4 had a privacy architecture whose central claim was contradicted by its own code: the deck said the server never sees plaintext, and the server decrypted every client's full 44,164-parameter update on every round. That is now an enforced invariant rather than a claim.

25.5 Why the headline numbers moved — in both directions

Fixing the defects changed the reported accuracies. It is worth being precise about why, because the changes go both ways and each direction means something different.

Dataset	Month 4	Month 5	Direction	Why
can_vtc	1.0000	1.0000	not comparable	4-class filename labels became a 2-class signature-labelled task
car_hack	0.7383	0.9977	UP	The old task was incoherent — every window of the DoS capture had to be called 'DoS', including the ~76% that contained no injection. With correct per-message labels the task is well-posed and solvable
cicids	0.9818	0.9555	DOWN	The leaky random split was inflating it. This drop is the leakage being removed — the number got smaller and truer
veremi	0.5290	0.4839	not comparable	Formulation changed entirely; both old and new are near chance, for different reasons (see 25.3)

The Car-Hacking result is the one to understand. Accuracy rose from 0.74 to 0.998 *because* the labels were fixed, not despite it. Under filename labelling the model was being asked to call quiet, injection-free stretches of the DoS capture "DoS" — a task with no correct answer. Removing that contradiction is what the jump measures.

Caveat. The Month-4 column is shown for direction only. Those runs used filename labels, a leaky split, simulated encryption and a single seed, and in some cases different row caps — they are not a valid baseline and must not be quoted as one. They are retained in Results/month4_legacy_INVALID/ purely for provenance.

26. Cryptographic measurements

Measured on the project machine; reproducible with `scripts/benchmark_ckks_profiles.py`.

Configuration	Total bits	Levels	Ciphertext	eq. 11 time	2nd ct x ct
N=8192, [60,40,40,60]	200	2	326.6 KB	18.0 ms	fails
N=8192, [60,40,40,40,60]	240	-	REJECTED	-	-
N=16384, [60,40,40,40,60]	240	3	836.1 KB	67.7 ms	works
N=16384, [60,40,40,40,40,60]	280	4	1028.6 KB	92.5 ms	works

What this means. At N=8192, 128-bit security permits at most **218 bits** of total coefficient modulus. A 240-bit chain is therefore **rejected outright** — buying a third multiplicative level requires doubling the ring dimension.

26.1 Cost of a full round, measured end to end

The table above is *per ciphertext*. What actually matters operationally is the cost per **client update** — the whole 44,164-parameter vector, which is 11 ciphertexts at N=8192 but only 6 at N=16384, because the larger ring packs twice as many slots. Measured with 7 active clients:

Per client update	practical (N=8192)	strict (N=16384)	Ratio
Ciphertext on the wire	3.51 MB	4.90 MB	1.40x
Expansion over float32	21.2x	29.6x	1.40x
Encrypt	41.9 ms	55.3 ms	1.32x
Encrypted distance (eq. 11)	228.6 ms	464.0 ms	2.03x
Homomorphic aggregation	52.0 ms	79.9 ms	1.54x
Decrypt aggregate	9.0 ms	13.0 ms	1.45x

A correction worth stating. Comparing single ciphertexts suggested strict FheFL would cost 2.56x the bandwidth. Measured per *client update* it is only **1.40x** — because doubling the ring also doubles the slots per ciphertext, so the update needs 6 ciphertexts instead of 11. The distance computation roughly doubles rather than quadrupling. **Strict FheFL is materially more affordable than the per-ciphertext figure implies**, which strengthens the case for implementing it in Month 6.

Do not confuse the two distance figures. The 18.0 ms in the table above is one ciphertext; the ~231 ms here is a full 11-ciphertext client update. Both are correct and they measure different things.

Measured CKKS Profile Trade-off — why strict FheFL costs a larger ring

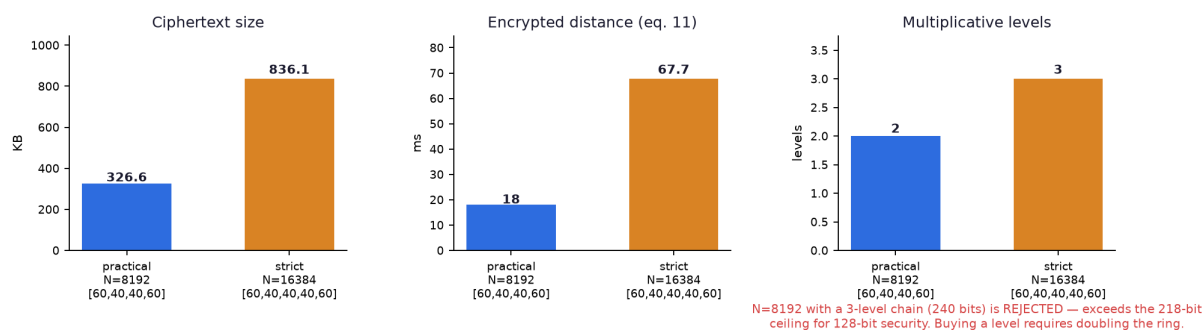


Figure 6 — Measured cost of the two disclosure profiles.

A correction. The Month-5 audit initially predicted that the modulus chain would have to be extended for eq. 11 itself. Measurement shows eq. 11 *does* fit at N=8192; the ceiling binds only on the second multiplication. The correction is recorded here rather than silently applied.

Part V — Current status

27. Component status

Completed

- Unified 46-D feature extraction across four heterogeneous datasets
- Per-message ground-truth labelling (Car-Hacking R/T flag, CAN-VTC 0x000 signature, VeReMi attack column)
- Leakage-free group-aware train/val/test splitting, machine-verified
- ChebyKAN classifier, 44,164 parameters
- Dirichlet non-IID sharding ($\alpha = 0.3$)
- Federated training loop with client sampling
- Real CKKS encryption of client updates (TenSEAL)
- FheFL encrypted-domain robust aggregation (eq. 6–12)
- Multi-key additive key sharing with per-round dropout tolerance
- ECDH-P256 + AES-256-GCM + Ed25519 authenticated transport
- Machine-checked privacy invariant (zero individual-update decryptions)
- Two verification suites, 105 assertions
- Threat model, security specification, library evaluation documents
- Measured CKKS cost for both disclosure profiles

Partially completed

- **Multi-key CKKS** — homomorphic arithmetic is real; key management is an algebraic model, because TenSEAL exposes no multi-key API. Needs the OpenFHE migration.
- **Byzantine robustness** — works against large-magnitude poisoning; untested against low-magnitude persistent attackers.
- **Library evaluation** — selection complete; cross-compilation not performed.
- **VeReMi** — per-beacon formulation works; sender-grouped windowing awaits data re-acquisition.
- **Experimental scale** — 10 clients / 5 rounds against a roadmap target of 50–200 / 200–300.
- **Ciphertext serialization** — sizes measured; no V2X transport protocol yet.

Not started

- Cross-compilation to an embedded target
- On-hardware performance benchmarking
- SIMD packing and encoding module
- NTT kernel acceleration
- Model quantization
- On-device inference engine / embedded IDS executable
- V2X communication protocol

- Client-level differential privacy
- Hardware-in-the-loop validation
- Power-consumption audit

28. Known limitations

Limitation	Impact	Mitigation path
Multi-key layer is an algebraic model, not a cryptographic implementation	The system must not be described as full multi-key CKKS	OpenFHE migration (Month 6)
Low-magnitude persistent poisoning is unaddressed	A patient attacker can bias the model while scoring a high non-poisoning rate	Experiment scheduled; may require a contribution-history term
CAN-VTC reduces to flood detection	Near-perfect scores are not evidence of generalisation	Report Car-Hacking as the primary CAN benchmark
CAN-VTC Fuzzy and Impersonation captures excluded	4-class problem reduced to 2 classes	Re-acquire raw data with injection ground truth
Noise flooding defaults to off	IND-CPA ^D exposure if a deployment inherits the default	Make the server refuse to start without it in a deployment profile
Round number not bound into AEAD associated data	A captured frame could be replayed in a later round	One-line fix, scheduled
Bandwidth: 3.59 MB per vehicle per round	Does not fit V2X message budgets (~2.7 KB per DSRC frame)	Modulus switching, sparsification, hierarchical RSU aggregation
Everything runs on a laptop CPU	No evidence the system is feasible on automotive hardware	Order target board; cross-compile

29. LG deliverable status

Period	Deliverable	Status
M1–2	Security specification report	Delivered (Month 5)
M1–2	Baseline performance benchmarking on hardware	Harness + host baseline delivered; target-board run blocked on procurement
M1–2	Threat model document	Delivered (Month 5)
M1–2	Baseline architecture design	Delivered
M3–4	Library evaluation & cross-compilation	Evaluation delivered; cross-compilation blocked on hardware
M3–4	Optimized packing & encoding module	Delivered (Month 5) — 29.6% lossless bandwidth reduction
M3–4	High-performance NTT implementation	Reference delivered + validated (11.22x); ARM/NEON kernel blocked on hardware
M3–4	Preprocessing engine	Delivered
M3–4	Model accuracy report	Delivered (Month 5, on corrected data)
M5–6	Ciphertext serialization & communication protocol	Measurements only
M5–6	Quantized model weights	Not started
M5–6	Embedded IDS executable	Not started
M7–12	Homomorphic aggregator (verified)	Ahead of schedule — implemented in Month 5

Overall position. 7 of the 8 deliverables due by end of Month 4 are now complete or substantially complete, up from 5 of 8 before the Month-5 catch-up work. The cryptographic and machine-learning tracks are healthy and the aggregation work is ahead of schedule.

The residual is one thing: no hardware has been ordered. Both remaining partial items — the ARM/NEON NTT kernel and the target-board baseline — are blocked on exactly that and on nothing else, and months 7–12 (hardware-in-the-loop, power audit) depend on them. Ordering a board is one hour of work and it is the highest-leverage action available.

Part VI — What to do next

Full detail in `Documentation/12_next_steps.md`. Priority order:

Priority	Task	Effort	Why first
P0.1	Order target hardware (NXP S32G + Raspberry Pi 5 as interim)	1 h	Blocks the entire embedded track
P0.2	Bind round number into AEAD associated data	1 h	Closes the replay threat
P0.3	Enforce noise flooding in deployment profiles	2 h	Closes IND-CPA ^D exposure
P0.4	Implement the strict disclosure profile (currently raises <code>NotImplementedError</code>)	3 d	Removes the one remaining per-round disclosure; may be cheaper after P1.1
P1.1	Migrate to OpenFHE	12 d	TenSEAL cannot express multi-key CKKS — the single most important item
P1.2	Adopt threshold (t-of-n) key sharing	in P1.1	Removes dropout fragility structurally
P1.3	Low-magnitude poisoning experiment	3 d	Tests the known blind spot
P1.4	Scale to 50 clients / 50 rounds	4 d	Moves toward the roadmap target
P1.5	Re-acquire raw VeReMi with sender IDs	2 d	Unlocks sender-grouped windowing
P1.6	Client-level differential privacy	3 d	The one roadmap mechanism still missing
P2.x	Embedded track: cross-compile, NTT, packing, quantization, V2X protocol	~40 d	The schedule debt
P3.x	Integration, HiL validation, power audit, final report	months 10–12	Programme close

Before publication or submission. Do not claim full multi-key CKKS until the OpenFHE migration lands. Report CAN-VTC as flood detection. Never re-report the Month-4 numbers — they are retained in `Results/month4_legacy_INVALID/` for provenance only. Run the low-magnitude poisoning experiment before claiming Byzantine robustness.

Part VII — Reproduction and verification

Folder layout

Folder	Contents
<code>Documentation/</code>	This PDF and all twelve markdown documents
<code>Complete Source Code/NewModel_active/</code>	The current, runnable system
<code>Complete Source Code/Model_gen1_archived/</code>	Generation 1 (KANConvNet) — reference only
<code>Complete Source Code/ArchitectureViewer_web/</code>	React architecture visualiser
<code>Newly Added - Month 5/</code>	Only what was added or changed in Month 5, plus a changelog
<code>Month 1-2 Foundation/</code>	Decks, reports and Phase-1 checkpoints from months 1–2
<code>Month 3-4 FHE and Architecture/</code>	FHE study, CKKS mathematics, MamKANformer and ChebyKAN decks
<code>Dataset/</code>	Manifest and provenance — the 8.6 GB corpus is referenced, not duplicated

Folder	Contents
Models/	Trained checkpoints
Experiments/	Benchmark and sweep scripts, test suites
Results/	Month-5 JSON results, and the Month-4 legacy results marked INVALID
Diagrams/	All figures in this document, as PNG and SVG

Running it

```
cd "Complete Source Code/NewModel_active"

python tests/test_data_pipeline.py    # 25 setup-correctness assertions
python tests/test_crypto.py          # 32 cryptographic assertions

python train_federated.py --dataset car_hack --backend ckks --seeds 2025 2026 2027
python train_federated.py --dataset car_hack --backend none --seeds 2025 2026 2027

python scripts/benchmark_ckks_profiles.py
python scripts/run_all_experiments.py
python scripts/generate_diagrams.py
python scripts/build_master_pdf.py
```

Always run both test suites after touching the data or crypto layers. They assert that the experimental setup is sound — no label leakage, no split leakage, no individual-update decryption. If they fail, every metric the pipeline produces is meaningless.

Environment

Component	Version
Python	3.12.0
PyTorch	2.13.0+cpu
TenSEAL	0.3.16
NumPy / pandas / scikit-learn	2.4.1 / 2.3.3 / 1.9.0
cryptography	46.0.4
Platform	Windows 11, CPU only